Coursework Report
Programming for Data Science
ST2195

Trixie Grace Mok

220659213

University of London

March 31, 2024

Part 1

Introduction

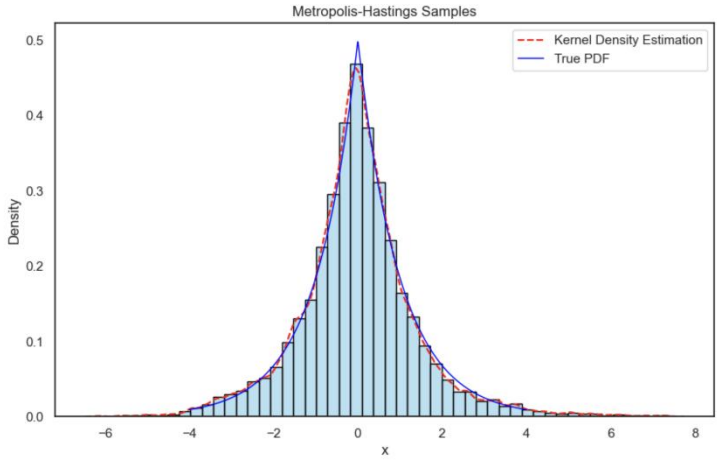
The first part of this report introduces the Markov Chain Monte Carlo Algorithm. Its purpose is to demonstrate a simulation of random samples using both R and Python to show a probability distribution. Used in computational statistics, it gives deeper insight and flexibility into underlying statistical model and navigation for complex distributions.

Method

First, we loaded the necessary libraries for both R and Python. Then the question provides a probability density function f(x) = 1 2 exp (−|x|) which we first initialise as a function where x takes the value. The next function initialised is the Random Walk Metropolis or as mentioned in the .Rmd and .ipynb files , Metropolis-Hastings algorithm function, which calls the probability density function at x*, it takes the value of N and s to generate samples.

The following outputs will alternate between Python and R as the outputs for both are the same and can be found in the attached .Rmd and .ipynb files.

Part (a) Python

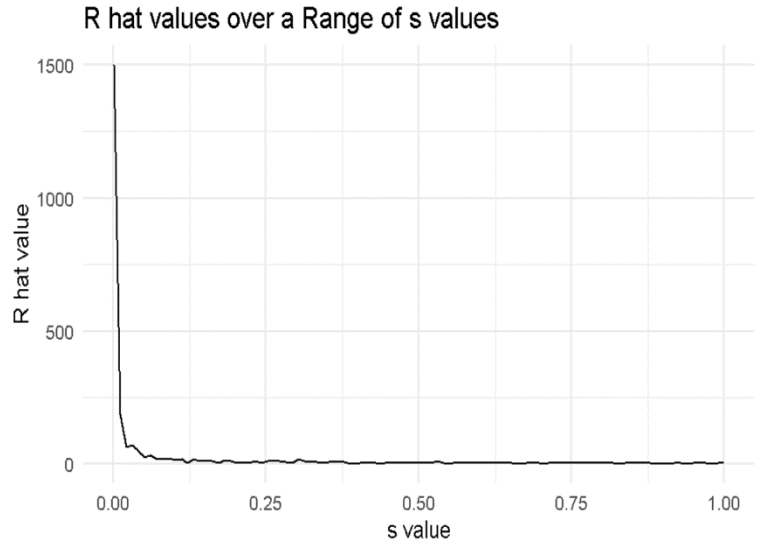



```
Sample mean: -0.008788528672679995
Sample standard deviation: 1.389513039927941
```

In (a), we were instructed to take the values of N = 1000 and s = 1 and construct a histogram and kernel density plot in one figure to display the generated samples and report the sample mean and standard deviation.

The histogram represents the frequency of the sample values. The nature of the given distribution $f(x)=21 e-|x|$ displays a bell-shaped centred at 0. The bars ascending from the peak represents the exponential decay of the distribution's extremities. The Kernel Density plot provides the estimates of the probability density function and the True PDF displays the actual probability density function.

Here we see that the Kernel Density plot and the True PDF are closely aligned which proves that the Metropolis-Hastings algorithm has effectively sampled from the target distribution.



Part (b) R

In (b) we converge Metropolis-Hastings algorithm using R hat statistic. The simulation recalls the probability density function, creates a new Metropolis-Hastings algorithm function with a random generated initial value as an input, and a new function that computes the R hat. We adjusted the scale for parameter s values from 0.001 to 1 over 100 values and create four chains (J) for each value with chain length of 2000 iterations (N). The R hat generated below 1.05 thus signifying the convergence.

Additional Notes

As the outputs for the graphs may differ in R and Python, we have adjusted some parameters for instance in Part a Python code, the Kernel Density Estimation line was not aligned with the histogram as it would in R, thus an adjusted bandwidth in the code. As for Part b, the y-axis of 0 was not at the margin, thus we specified the placement in ylim.

Part 2

Introduction

The second part of the report will analyse flight data from Harvard Database. Its purpose is to give insights on airline companies to better understand or forecast customer demands. Part 2 will choose the data of ten years from 1999 to 2008, airports, carriers, and plane data by taking the files 1999.csv., 2000.csv, 2001.csv, 2002.csv, 2003.csv, 2004.csv, 2005.csv, 2006.csv, 2007.csv, 2008.csv, airports.csv, carriers.csv, and plane-data.csv.

Method

After loading the R and Python libraries we created each table for each csv files. To answer each question, we looped through each year (or tables) so the data would not exceed the RAM limit. Within each loop we filtered out flights that were cancelled and diverted, removed duplicate rows, and filled missing values in columns with the last observation carried forward. As the outputs for each questions produced multiple graphs for each year, we have selectively chosen few of the graphs for simplicity as the reader may assess each graph within the Python notebook and R markdown files.

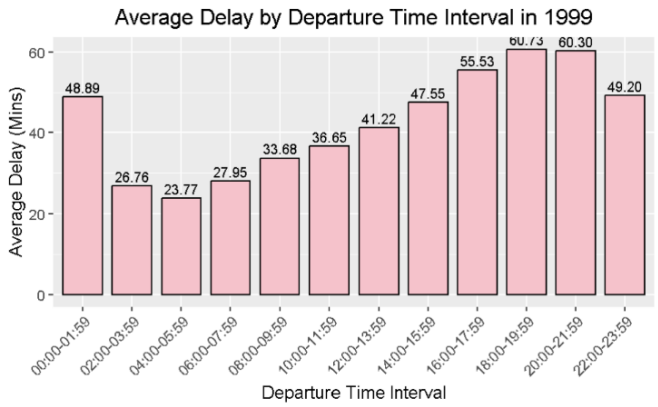
(a) What are the best times and days of the week to minimise delays each year?

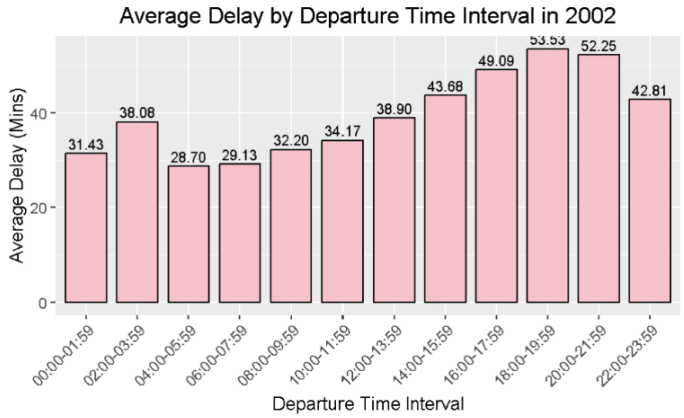
For this question, we did it in three parts: best time of day, best day of week, and best month of year. To determine the delays, we aggregate all ArrDelay, DepDelay, CarrierDelay, NASDelay, SecurityDelay, LateAircraftDelay, and WeathDelay into a total of delays. Then within each loop we filtered flights with the total delay by filtering total delays greater than 0.

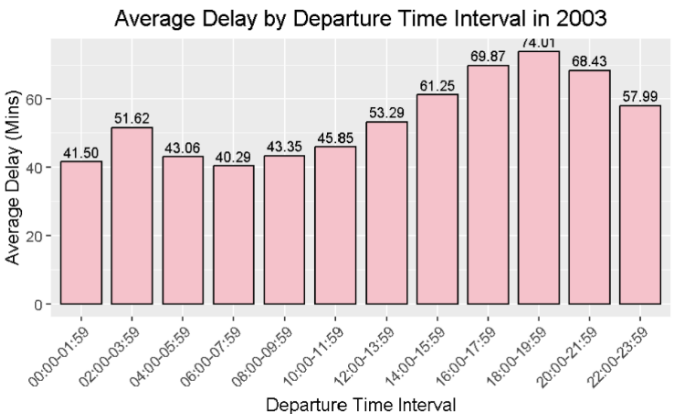
In the first part: **Best Time of the Day**, we made departure time intervals of two-hour constant time frames to display the outputs in a time frame interval. The output produced ten graphs to analyse each year.

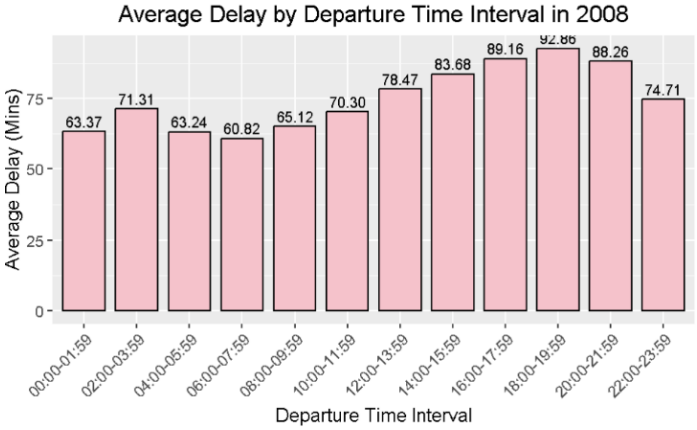
Based on the bar graphs, in 1999 to 2002 the best time to travel was between 4:00AM to 5:59 AM with an average delay cap of 28.70 minutes. From 2003 to 2008, the best time to travel was from 6:00 AM to 7:59AM. On the other hand, the time between 6:00pm to 7:59pm experiences the most delays with an average delay range of 53.53 to 112.49 minutes.

These are the following R bar charts for the best departure times for each year:





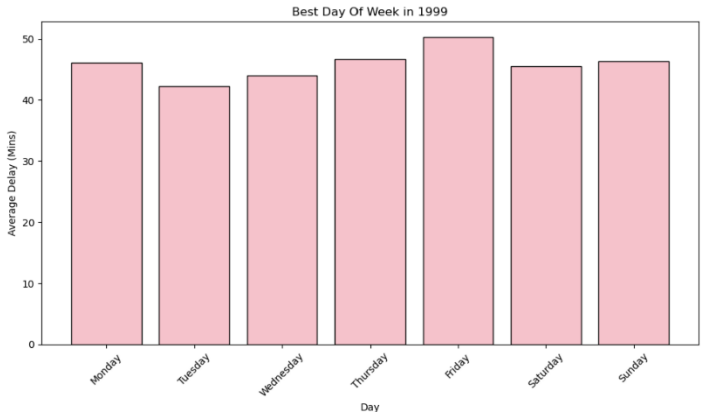


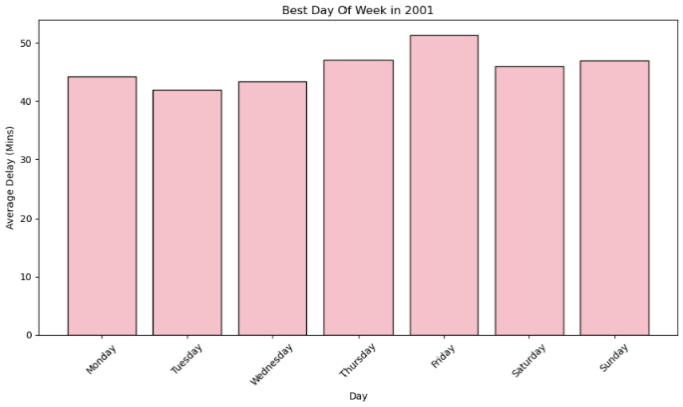


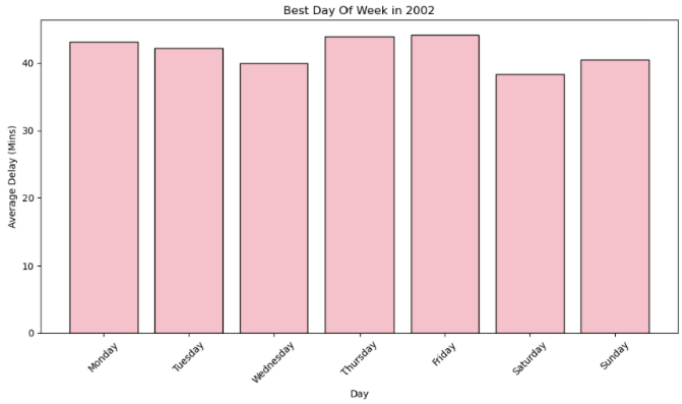
The second part will access the best day of the week, the output below represents Python graphs.

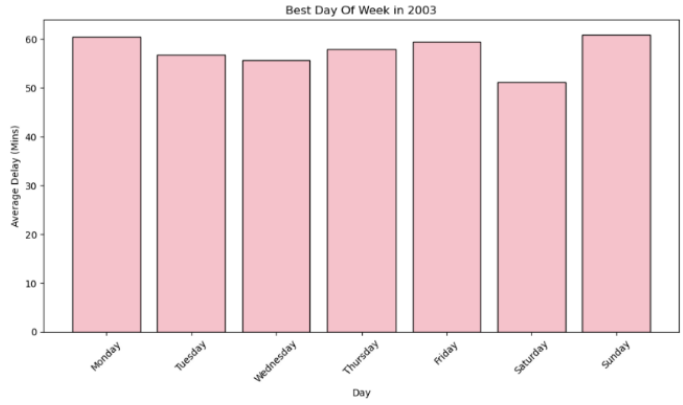
Best Day of the Week

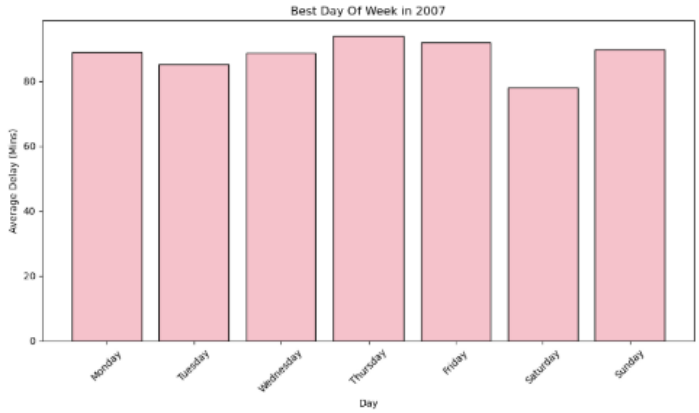
In years 1999 to 2001, the best day to fly was Tuesday. In contrast, Friday had the most delays among those years. In 2002, the best day was Wednesday and the peak delays were on Thursday and Friday. For years 2003 to 2007, the best day to fly was Saturday. Lastly, in 2008, the best day to fly was Wednesday with a prominent peak in delays on Friday.

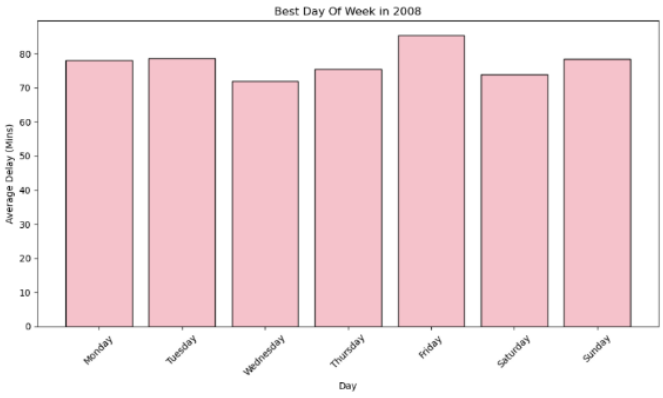








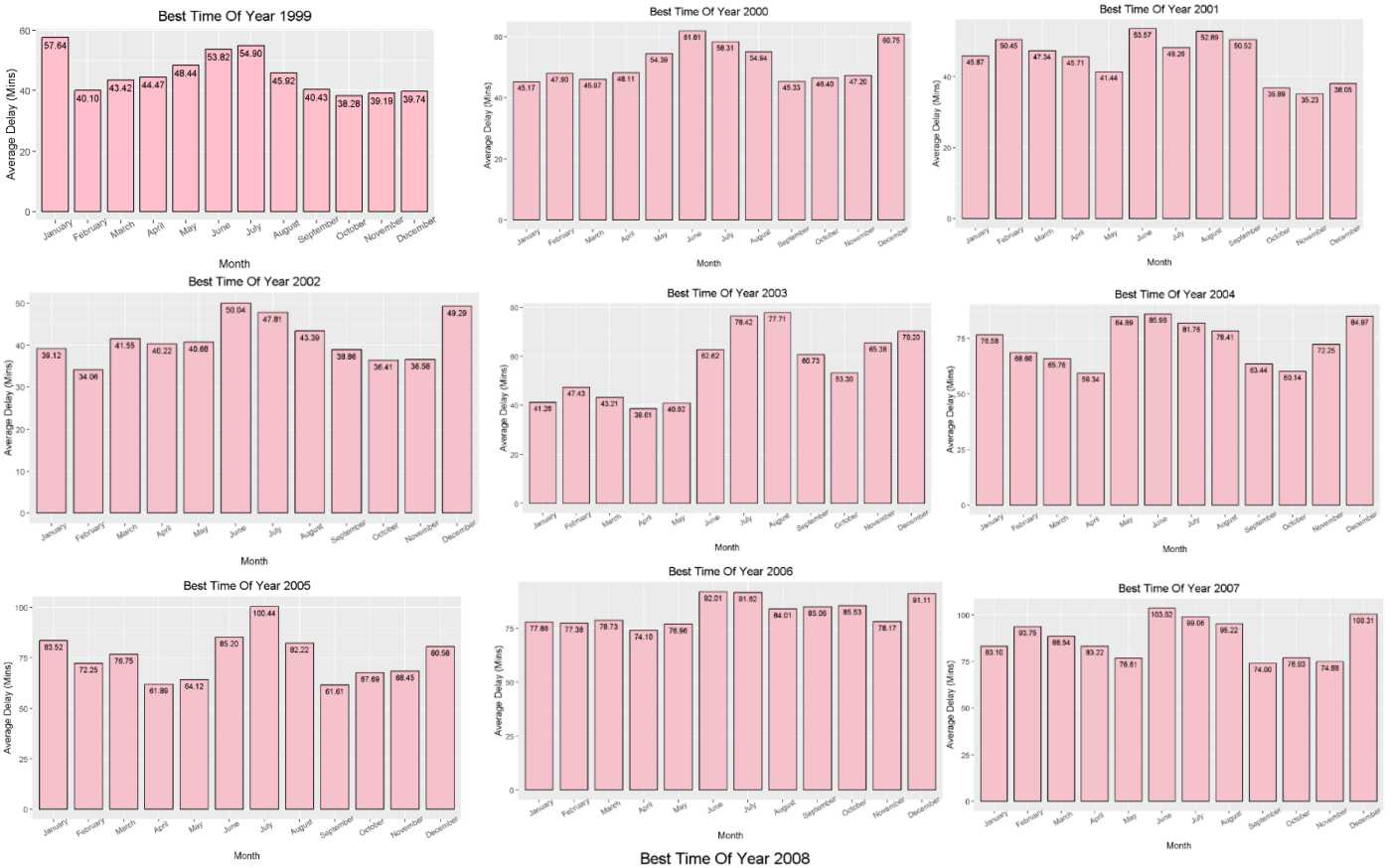




Best Month of the Year

The last part of the question will answer the most ideal month to fly within the specific year. The graphs below are outputs of R code.

In 1999 the best month to fly was October and January had the most delays. 2000, January had the least delays and June had the most. 2001, November was the best month to fly and June was the worst. 2002, February had the least delays and June had the most. In 2003, the best month was April and the month with most delays was August. 2004, April was the best month and June the worst month to fly. 2005, September was the best month and July had the most delays. In 2006, the best month to fly was April and the worst was June. 2007, September had the least delays and June, the most. Lastly 2008 data for months April to December was empty thus we took the average delays and filled up those months. Based on the inaccurate data, we still can access that among the months' average delays, February was the worst day to fly.



*Note: For better viewing please refer to the R code output. *

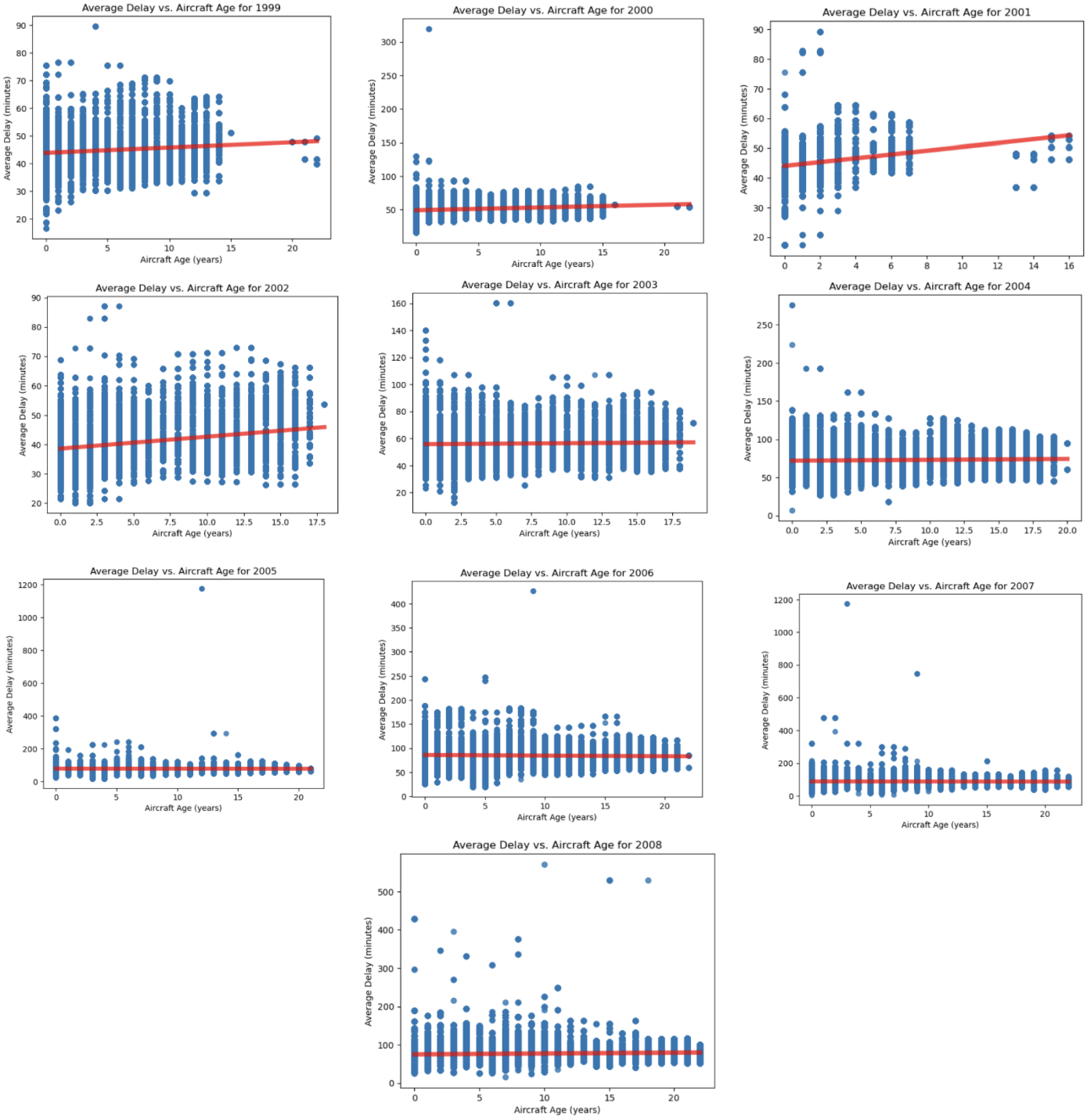
The following table summarises the findings for Part 2 (a)

What are the best times and days of the week to minimise delays each year?

Years	Time	Day	Month
1999	04:00-05:59	Tuesday	October
2000	04:00-05:59	Tuesday	January
2001	04:00-05:59	Tuesday	November
2002	04:00-05:59	Saturday	February
2003	06:00-07:59	Saturday	April
2004	06:00-07:59	Saturday	April
2005	06:00-07:59	Saturday	September
2006	06:00-07:59	Saturday	April
2007	06:00-07:59	Saturday	September
2008	06:00-07:59	Wednesday	-

Overall, the best time, days and moth of the year to minimize the delay for travellers would be the days with the least average delay therefore the following data chosen reflects as such. To analyse each trend, in time, most people flew in noon since most people would not wake up so early to catch a flight. As for the days, there is a trend that most people flew on Friday thus the days with the least delays were Tuesday, Saturday, and Wednesday. Lastly for the months, the shape of the graph had a peak during Summer as most people would go for holiday.

2(b) Evaluate whether older planes suffer more delays on a year-to-year basis (Python)



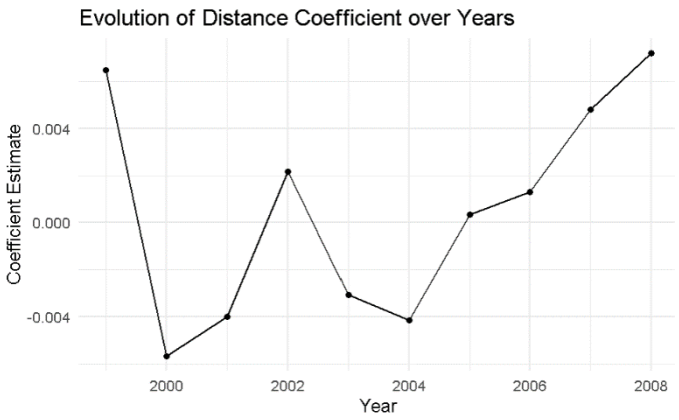
In year 1999, the line of best fit shows that younger planes (from 0 years) had a lower delay average than older planes (of 20 years) however the scatterplots show more delays than older planes as the there are more blue points on the left side of the graph. This trend is present at years 1999 to 2001. In years 2002 to 2004 the scatterplot points are more evenly dispersed. In 2002, the line of best fit for 2002 has a steeper incline showing that older planes had more delays than younger planes for the other years (2003 and 2004) the incline is not as prominent. From years 2005 to 2008 the line of best fit still shows a small incline but in ears 2006 and 2008, there were more points dispersed as a opposed too 2005 and 2007.

Based on the line of best fit for all graphs, older planes did experience more delays however it is important to note that the numbers of the y axis progressively increase as the years go by where in year 2008 the data averages at 100 minutes of average delay whereas in 1999, the average is at 44 minutes. Therefore, the line of best fit may not be the best indicator that older planes experienced more delays than newer ones.

2(c) For each year, fit a logistic regression model for the probability of diverted US flights using as many features as possible from attributes of the departure date, the scheduled departure and arrival times, the coordinates and distance between departure and planned arrival airports, and the carrier. Visualize the coefficients across years.

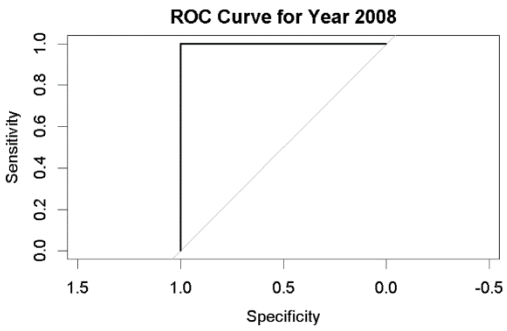
We created multiple graphs to display the logistic regression and its coefficients. In the First graph (Figure 1), we selected Distance as a predictor and the y-axis is the coefficient estimate for Distance which measure the change in log odds of a flight being diverted for increase in unit of distance. In years where the coefficient estimate is higher than 0, it proposes that longer flight distances increase the probability of a flight being diverted. Coefficients below 0 would indicate that longer flight distances would decrease the probability of diversion of flights.

Figure 1



Thus, the trend in early years around 2000 showing a dip displays that the particular year was less likely to be diverted. The following year 2001 had a rise that showed it was slightly more likely than 2000 although still unlikely as the coefficients remain a negative. Therefore, for years 1999, 2002, 2005 onwards experienced likely diverted flights as distance increases (because coefficients are above 0 and would increase).

Figure 2



In Figure 2 we created ROC curves for each year which showed that the logistic regression did work for all years.

In figure 3, we iterated each coefficient selected for logistic regression plotting. Though coefficients: Year, Month, DayofMonth, CRSDepTime, CRSArrTime, and UniqueCarried did not have an output. Only graphs for Distance, Origin_lat, Origin_long, Dest_lat, and Dest_long did.

Figure 3

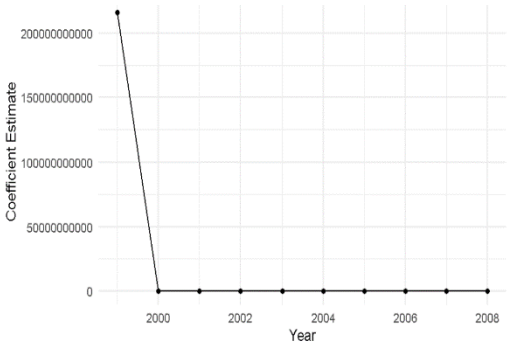


Figure 5 plots all coefficients in one graph the analysis did not plot very well for analysis as the point points converged at 0. The coordinates are not good indicators as it displayed outputs for each longitude and latitude. Regardless, the graph showed that Origin_lat was below 0 in 1999, meaning that as the origin's latitude increases, diverted flights were unlikely, this applies for dest_lat and dest_long as well. In 1999 Origin_long was well above zero which indicated that as the longitude of the origin increases, the likelihood of diversion was high.

The last Figure (6) is a snapshot of the logistic regression for diverted flights each year. The Coefficients only mentioned Month, DayofMonth and CRSDepTime. On the rightmost column is p values, these values are high (near 1) meaning that the coefficients on the leftmost column are not ideal predictors for the logistic regression. This explains why only a few coefficients could be plotted.

Figure 4

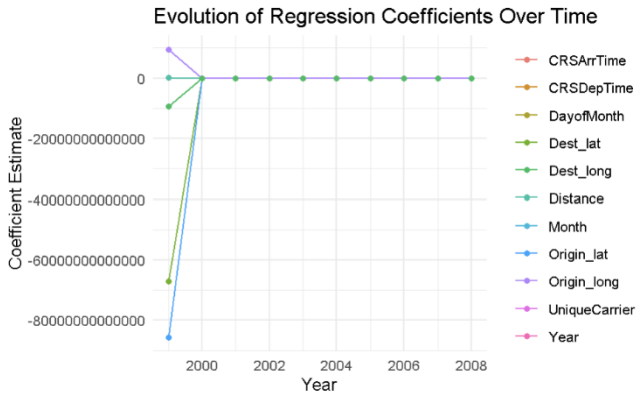


Figure 5

```
call:
glm(formula = formula, family = binomial(), data =
year_flight_processed)

Coefficients: (14 not defined because of singularities)
                    Estimate     Std. Error z value Pr(>|z|)
(Intercept)          -19.99773   718147.52185   0.000    1.000
Month2                 0.52406    27380.09592   0.000    1.000
Month3                 7.45327    25067.65554   0.000    1.000
Month4                -3.03959    30096.46831   0.000    1.000
DayofMonth2           10.88548    64755.38952   0.000    1.000
DayofMonth3           -8.92238    87356.42829   0.000    1.000

CRSDepTime919        -51.59346   532441.50665   0.000    1.000
CRSDepTime920        -36.64474   403294.27128   0.000    1.000
```